

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Navegación entre pantallas: inicio de sesión y registro con Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	03	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	19/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
EJERCICIOS/PROBLEMAS PROPUESTOS - Link Repositorio (Propuesta 1): https://github.com/shanccom/IDNP_Repo/tree/main/lab_03_Navbar - Link Repositorio (Propuesta 2): https://github.com/DeniseHuacani/IDNP-LABORATORIO/tree/main/lab03 Propuesta 1 Ejercicio 1: Agrega una tercera pantalla, HomeScreen(usuario), registrada en el NavHost con la ruta "home", que reciba el nombre del usuario como argumento de navegación (por ejemplo <code>navController.navigate("home/usuario")</code>) y lo muestre en el mensaje "Bienvenido". En este ejercicio se agrega una tercera pantalla a la aplicación de login/registro. La pantalla, llamada HomeScreen(usuario), se registra en el NavHost con la ruta <code>home/{usuario}</code> y recibe el nombre del usuario como argumento de navegación para mostrarlo en un mensaje 1. Definir la lista de cuentas registradas Para validar el ingreso se mantiene una lista global de cuentas. Por ahora contiene el usuario admin con contraseña 1234. Cuando un usuario se registra, la pareja usuario to password se agrega a esta lista: <pre>val cuentasRegistradas = mutableListOf("admin" to "1234")</pre> 2. Registrar la ruta <code>home/{usuario}</code> en el NavHost Dentro del NavHost, cuya ruta inicial es login, se registran tres pantallas: login, registro y <code>home/{usuario}</code>. La ruta de home recibe el argumento usuario de tipo StringType, declarado con <code>navArgument</code>. El valor se obtiene desde <code>backStackEntry.arguments?.getString("usuario")</code> y se pasa al Composable HomeScreen:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
composable( route = "login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            navController.navigate( route = "home/$usuario")
        },
        onIrARegistro = {
            navController.navigate( route = "registro")
        }
    )
}
```

Figura 1: Registro de la ruta home/{usuario} con su argumento de navegación

3. Navegar hacia HomeScreen al iniciar sesión

Cuando las credenciales ingresadas coinciden con alguna cuenta registrada, LoginScreen invoca onLoginExitoso(usuario). En MainActivity esa función navega a la ruta home pasando el usuario como argumento de la ruta:

```
composable("login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            navController.navigate("home/$usuario")
        },
        onIrARegistro = {
            navController.navigate("registro")
        }
    )
}
```

4. Crear el Composable HomeScreen

HomeScreen recibe el parámetro usuario: String y lo muestra en un Text con el mensaje "Bienvenido \$usuario":

```
1 Usage
@Composable
fun HomeScreen(usuario: String) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding( all = 24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Bienvenido $usuario",
            style = MaterialTheme.typography.headlineSmall
        )
    }
}
```

Figura 2: Función Composable HomeScreen(usuario)

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

5. Ejecutar y verificar

Al ejecutar el proyecto se muestra la pantalla de login. Al ingresar con el usuario admin y la contraseña 1234, la aplicación navega a la pantalla Home y muestra el mensaje de bienvenida con el nombre del usuario:

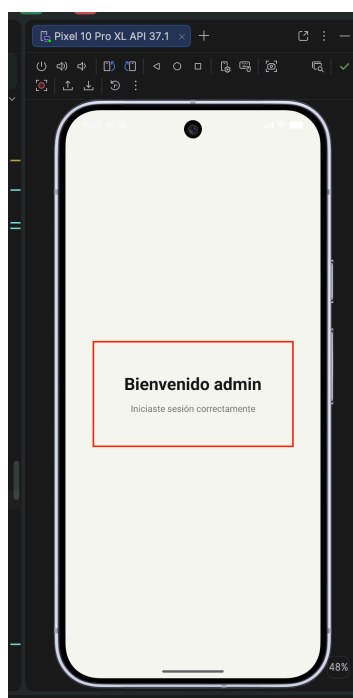


Figura 3: Pantalla de bienvenida con el nombre del usuario

Ejercicio 2: En RegistroScreen, al presionar “Aceptar”, guarda los datos ingresados (usuario y contraseña) en un archivo de texto plano cuentas.txt en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_APPEND), y regresa a LoginScreen mostrando un mensaje de confirmación.

En este ejercicio se persiste cada cuenta nueva en el archivo cuentas .txt del almacenamiento interno y se confirma la creación con el mensaje “Cuenta creada correctamente”.

1. Nuevos imports

Se agregan Context (para openFileOutput), Toast (mensaje de confirmación) y LocalContext (para obtener el contexto dentro del Composable):

```
import android.content.Context
import android.widget.Toast
import androidx.compose.ui.platform.LocalContext
```

2. Función guardarCuenta

La función abre cuentas.txt en modo MODE_APPEND y escribe una línea por cuenta con el formato usuario,password:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

1 Usage
fun guardarCuenta(context: Context, usuario: String, password: String) {
    val linea = "$usuario,$password\n"
    context.openFileOutput( name = "cuentas.txt", mode = Context.MODE_APPEND).use {
        archivo -> archivo.write( b = linea.toByteArray())
    }
}

```

Figura 4: Función guardarCuenta(context, usuario, password)

3. Cambios en RegistroScreen

Dentro del Composable se obtiene el contexto con `LocalContext.current` y el botón "Aceptar" ahora llama a `guardarCuenta(...)` antes de ejecutar `onRegistroExitoso()`:

```

val context = LocalContext.current

Button(
    onClick = {
        guardarCuenta(context, usuario, password)

        onRegistroExitoso()
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Aceptar")
}

```

4. Mensaje de confirmación en MainActivity

`onRegistroExitoso` muestra un Toast con "Cuenta creada correctamente" antes de regresar a `LoginScreen` con `popBackStack()`:

```

onRegistroExitoso = {
    Toast.makeText(
        this,
        "Cuenta creada correctamente",
        Toast.LENGTH_SHORT
    ).show()

    navController.popBackStack()
},

```

5. Verificación

Al presionar "Aceptar" por cada cuenta, las líneas quedan en el archivo `cuentas.txt`:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

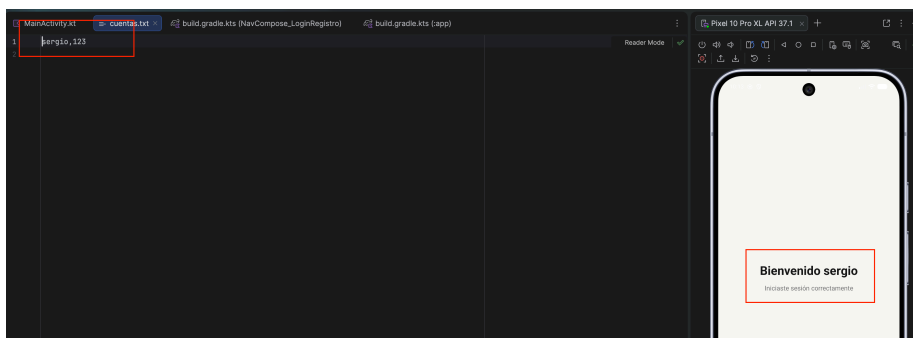


Figura 5: Archivo cuentas.txt con las cuentas guardadas

Ejercicio 3: En LoginScreen, al presionar “Ingresar”, busca una coincidencia en el archivo cuentas.txt (en lugar de la lista en memoria usada en el ejercicio resuelto). Si existe, navega a HomeScreen; si no, muestra el mensaje “Cuenta no encontrada”.

En este ejercicio la validación del ingreso ya no usa la lista en memoria cuentasRegistradas, sino que lee el archivo cuentas.txt para verificar si existe la cuenta usuario,password.

1. Función buscarCuenta

La función lee cuentas.txt con openFileInput y verifica si existe una línea que coincida exactamente con usuario,password. Si el archivo todavía no existe, devuelve false sin generar un error:

```
1 Usage
fun buscarCuenta(context: Context, usuario: String, password: String): Boolean {
    return try {
        val contenido = context.openFileInput(name = "cuentas.txt")
            .bufferedReader()
            .use { it.readText() }

        contenido.lineSequence().any { linea ->
            linea.trim() == "$usuario,$password"
        }
    } catch (e: Exception) {
        false
    }
}
```

Figura 6: Función buscarCuenta(context, usuario, password)

2. Cambios en LoginScreen

Dentro del Composable se obtiene el contexto con LocalContext.current:

```
val context = LocalContext.current
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

3. Botón “INGRESAR”

En lugar de buscar en cuentasRegistradas, el botón ahora llama a buscarCuenta(context, usuario, password). Si coincide, navega a home/\$usuario; si no, muestra “Cuenta no encontrada”:

```
val coincide = buscarCuenta(context, usuario, password)

if (coincide) {
    mensajeError = ""
    onLoginExitoso(usuario)
} else {
    mensajeError = "Cuenta no encontrada"
}
```

4. Verificación

Si la cuenta no existe en cuentas.txt, la pantalla de login muestra el mensaje de error:

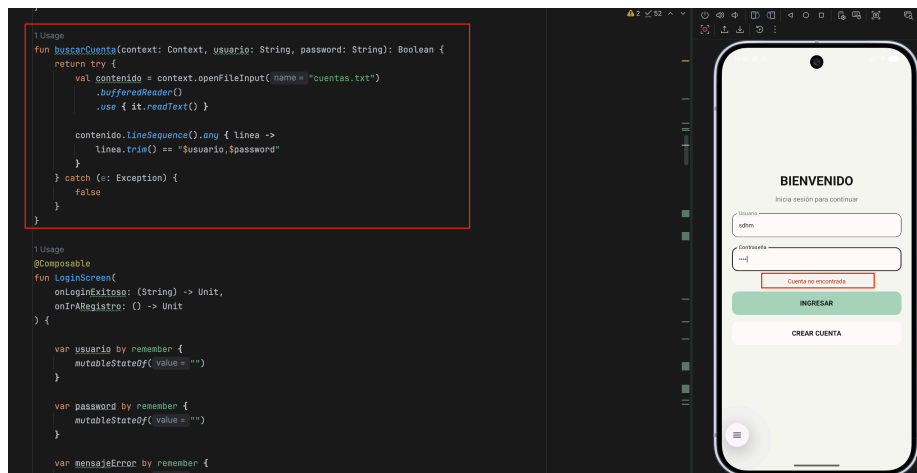


Figura 7: Mensaje “Cuenta no encontrada” al intentar ingresar con una cuenta inexistente

Ejercicio 4: (Reto opcional) Agrega validación de campos vacíos en ambos formularios, mostrando un mensaje de error si el usuario intenta enviar el formulario incompleto.

En este ejercicio se agrega validación de campos vacíos antes de guardar o buscar la cuenta: si falta algún dato, se muestra un mensaje y no se continúa con la operación.

1. Validación en RegistroScreen

Se agregó el estado mensajeError junto con su bloque de visualización (antes del botón “Aceptar”):

```
var mensajeError by remember {
    mutableStateOf("")
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

}

if (mensajeError.isNotEmpty()) {

    Text(
        text = mensajeError,
        color = MaterialTheme.colorScheme.error
    )

    Spacer(
        modifier = Modifier.height(8.dp)
    )
}

```

El botón “Aceptar” aplica la misma validación: solo guarda y navega atrás cuando ambos campos tienen contenido, y limpia mensajeError al registrar la cuenta correctamente:

```

Button(
    onClick = {
        when {
            usuario.isBlank() && password.isBlank() -> {
                mensajeError = "Completa todos los campos"
            }
            usuario.isBlank() -> {
                mensajeError = "El campo usuario es obligatorio"
            }
            password.isBlank() -> {
                mensajeError = "El campo contraseña es obligatorio"
            }
            else -> {
                val coincide = buscarCuenta(
                    context, usuario, password
                )
            }
        }
    }
)

```

Figura 8: Código de validación de campos vacíos

2. Verificación

Con la validación implementada, al dejar campos vacíos se muestra el mensaje correspondiente y no se realiza la búsqueda ni el guardado:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

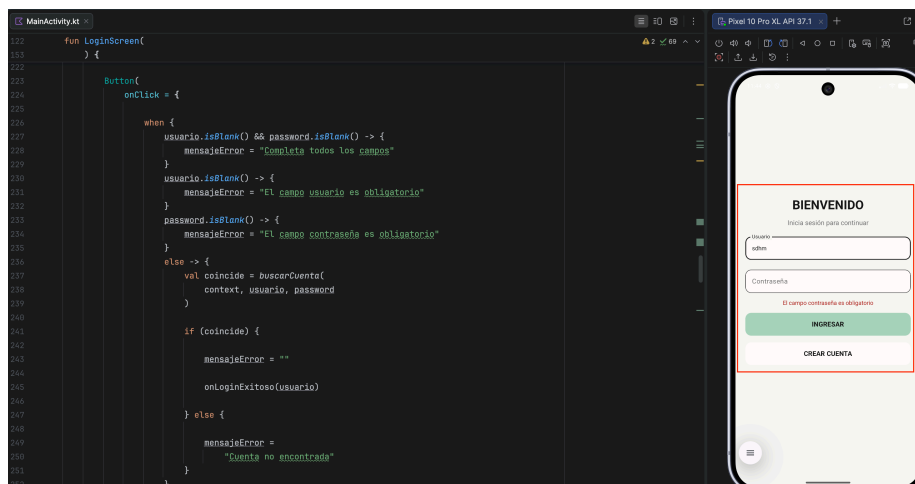


Figura 9: Mensaje de validación mostrado al enviar el formulario incompleto

Propuesta 2

Ejercicio 1: Agrega una tercera pantalla, **HomeScreen(usuario)**, registrada en el **NavHost** con la ruta “home”, que reciba el nombre del usuario como argumento de navegación (por ejemplo `navController.navigate(“home $usuario”)`) y lo muestre en el mensaje “Bienvenido usuario”.

En este ejercicio se agregó una tercera pantalla, que se muestra cuando el inicio de sesión es correcto. Se registró en el **NavHost** con la ruta “home/{usuario}”. La parte entre llaves es un **argumento de navegación**: se declara con `navArgument` y se lee en la pantalla destino mediante `backStackEntry.arguments?.getString(“usuario”)`.

Para navegar se construye la ruta con el valor real del usuario, es decir `navController.navigate(“home/$usuario”)`. Se aplicó `Uri.encode(usuario)` para que caracteres especiales (como /) no rompan la ruta. Además, se usó `popUpTo(“login”) { inclusive = true }` para eliminar la pantalla de login de la pila de navegación, de modo que al presionar el botón Atrás no se regrese al formulario de inicio de sesión.

Nota: el **HomeScreen** que existía en el código de partida era una copia de **RegistroScreen**; se reemplazó por la nueva versión.

1. Nueva pantalla HomeScreen:

```
@Composable
fun HomeScreen(usuario: String) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Bienvenido $usuario", style = MaterialTheme.typography.headlineMedium)
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

2. Registro de la ruta con argumento en el NavHost:

```
composable(
    route = "home/{usuario}",
    arguments = listOf(
        navArgument("usuario") { type = NavType.StringType }
    )
) { backStackEntry ->
    val usuario = backStackEntry.arguments?.getString("usuario") ?: ""
    HomeScreen(usuario)
}
```

3. Navegación desde el login (reemplaza el Toast anterior):

```
composable("login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            navController.navigate("home/${Uri.encode(usuario)}") {
                popUpTo("login") { inclusive = true }
            }
        },
        onIrARegistro = {
            navController.navigate("registro")
        }
    )
}
```

4. Ejecucion

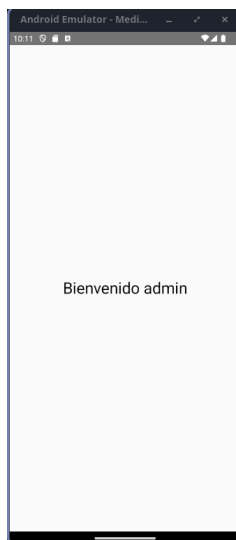


Figura 10: Ejecución del ejercicio 1

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

Ejercicio 2: En RegistroScreen, al presionar “Aceptar”, guarda los datos ingresados (usuario y contraseña) en un archivo de texto plano cuentas.txt en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_APPEND), y regresa a LoginScreen mostrando un mensaje de confirmación.

Para reemplazar la lista en memoria se creó la función guardarCuenta, que abre el archivo cuentas.txt con openFileOutput en modo Context.MODE_APPEND. Esto agrega el contenido al final del archivo sin borrar las cuentas anteriores, y el archivo se guarda en el almacenamiento interno privado de la aplicación. Cada cuenta ocupa una línea con el formato usuario,password.

En RegistroScreen se obtiene el Context con LocalContext.current. Al presionar “Aceptar” se guarda la cuenta, se muestra un Toast de confirmación y se llama a onRegistroExitoso(), que ejecuta popBackStack() y regresa a LoginScreen. El Toast sigue visible después del cambio de pantalla, por lo que el mensaje aparece sobre el login.

1. Función que escribe en el archivo:

```
const val ARCHIVO_CUENTAS = "cuentas.txt"

fun guardarCuenta(context: Context, usuario: String, password: String) {
    context.openFileOutput(ARCHIVO_CUENTAS, Context.MODE_APPEND).use { salida ->
        salida.write("$usuario,$password\n".toByteArray())
    }
}
```

2. Botón “Aceptar” de RegistroScreen:

```
val context = LocalContext.current // al inicio de RegistroScreen

Button(
    onClick = {
        val nombre = usuario.trim()
        guardarCuenta(context, nombre, password)
        Toast.makeText(
            context,
            "Cuenta \"$nombre\" registrada correctamente",
            Toast.LENGTH_SHORT
        ).show()
        onRegistroExitoso()
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Aceptar")
}
```

3. Ejecucion

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

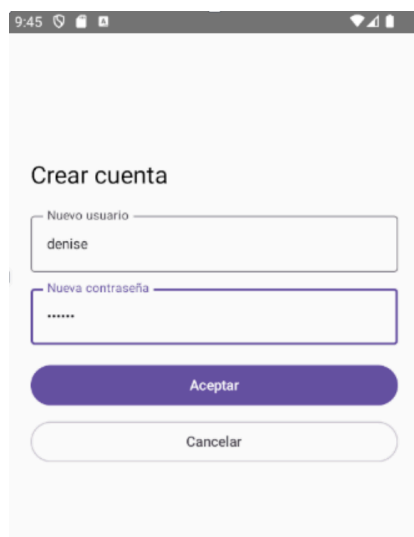


Figura 11: Ejecución del ejercicio 1

```
com...mple.navcompose_loginregistro E >>> SE GUARDÓ CORRECTAMENTE: denise,123456 <<<
```

Figura 12: Mensaje de guardado exitoso

Ejercicio 3: En LoginScreen, al presionar “Ingresar”, busca una coincidencia en el archivo cuentas.txt (en lugar de la lista en memoria). Si existe, navega a HomeScreen; si no, muestra el mensaje “Cuenta no encontrada”.

Se eliminó la lista cuentasRegistradas y se creó la función existeCuenta. Esta obtiene el archivo desde context.filesDir, verifica que exista (si nadie se ha registrado todavía, devuelve false) y lee todas las líneas con readLines(). Cada línea se separa con split(",", limit = 2); el parámetro limit = 2 permite que la contraseña contenga comas sin romper la lectura. Finalmente se compara usuario y contraseña con any { ... }.

En LoginScreen, si existeCuenta devuelve true se llama a onLoginExitoso(nombre), que navega a HomeScreen; de lo contrario se muestra el mensaje “Cuenta no encontrada” en color de error.

1. Función que busca en el archivo:

```
fun existeCuenta(context: Context, usuario: String, password: String): Boolean {
    val archivo = File(context.filesDir, ARCHIVO_CUENTAS)
    if (!archivo.exists()) return false

    return archivo.readLines().any { linea ->
        val partes = linea.split(",", limit = 2)
        partes.size == 2 && partes[0] == usuario && partes[1] == password
    }
}
```

2. Botón “Ingresar” de LoginScreen:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```
val context = LocalContext.current // al inicio de LoginScreen

Button(
    onClick = {
        val nombre = usuario.trim()
        if (existeCuenta(context, nombre, password)) {
            mensajeError = ""
            onLoginExitoso(nombre)
        } else {
            mensajeError = "Cuenta no encontrada"
        }
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Ingresar")
}
```

3. Ejecucion

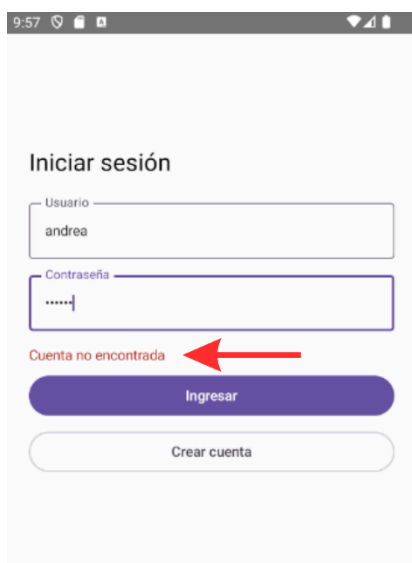


Figura 13: Validación de inicio de sesión con cuenta inexistente: "Cuenta no encontrada"

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

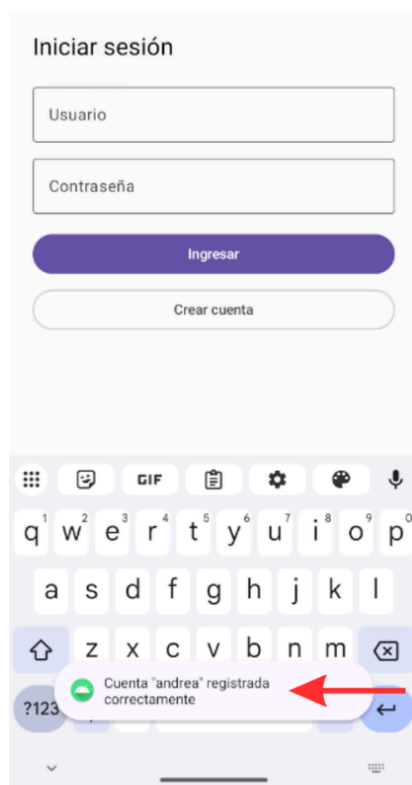


Figura 14: Registro de usuario y Toast de confirmación

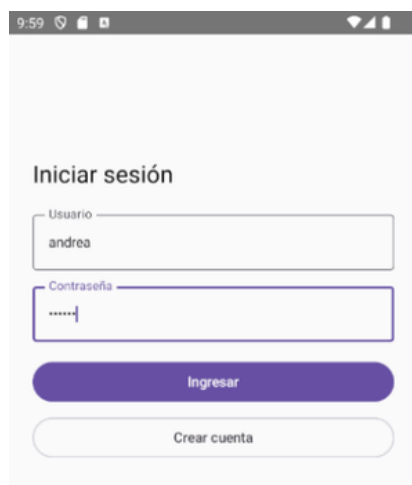


Figura 15: Inicio de sesión exitoso

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14



Figura 16: Inicio de sesión exitoso y navegación a HomeScreen

Ejercicio 4 (Reto opcional): Agrega validación de campos vacíos en ambos formularios, mostrando un mensaje de error si el usuario intenta enviar el formulario incompleto.

Se agregó validación con una expresión `when` en los botones de ambas pantallas. Antes de acceder al archivo se comprueba que el usuario y la contraseña no estén vacíos; si alguno lo está, se muestra el mensaje "Complete todos los campos" mediante la variable de estado `mensajeError`, que se dibuja en color de error. En `RegistroScreen` se creó además esta variable (no existía) junto con el bloque `if (mensajeError.isNotEmpty())` que muestra el texto.

Como la coma es el separador del archivo, en el registro se rechaza también un usuario que la contenga. Adicionalmente, a los campos de texto se les agregó `singleLine = true` para evitar que un salto de línea en la contraseña corrompa el formato del archivo.

1. Validación en `LoginScreen`:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

Button(
    onClick = {
        val nombre = usuario.trim()
        when {
            nombre.isEmpty() || password.isEmpty() -> {
                mensajeError = "Complete todos los campos"
            }
            existeCuenta(context, nombre, password) -> {
                mensajeError = ""
                onLoginExitoso(nombre)
            }
            else -> {
                mensajeError = "Cuenta no encontrada"
            }
        }
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Ingresar")
}

```

2. Validación en RegistroScreen:

```

var mensajeError by remember { mutableStateOf("") } // nuevo estado

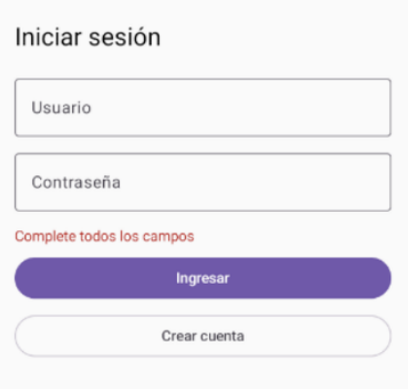
// ...debajo del campo de contraseña:
if (mensajeError.isNotEmpty()) {
    Text(mensajeError, color = MaterialTheme.colorScheme.error)
    Spacer(modifier = Modifier.height(8.dp))
}

Button(
    onClick = {
        val nombre = usuario.trim()
        when {
            nombre.isEmpty() || password.isEmpty() -> {
                mensajeError = "Complete todos los campos"
            }
            nombre.contains(",") -> {
                mensajeError = "El usuario no puede contener comas"
            }
            else -> {
                guardarCuenta(context, nombre, password)
                Toast.makeText(
                    context,
                    "Cuenta \"$nombre\" registrada correctamente",
                    Toast.LENGTH_SHORT
                ).show()
                onRegistroExitoso()
            }
        }
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Aceptar")
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

3. Ejecucion



Iniciar sesión

Usuario

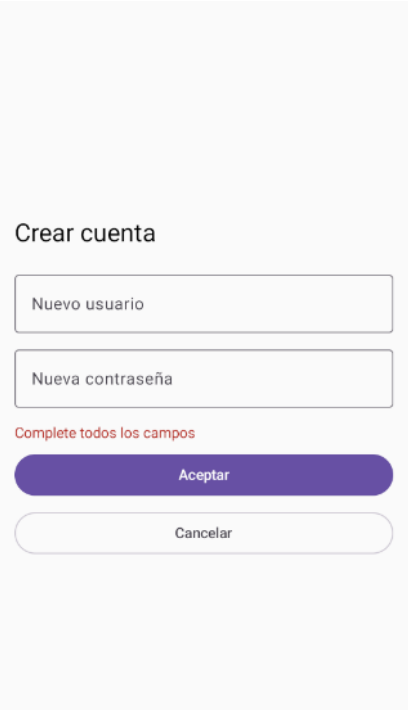
Contraseña

Complete todos los campos

Ingresar

Crear cuenta

Figura 17: Validación de campos vacíos en la pantalla de Login



Crear cuenta

Nuevo usuario

Nueva contraseña

Complete todos los campos

Aceptar

Cancelar

Figura 18: Validación de campos vacíos en la pantalla de Registro

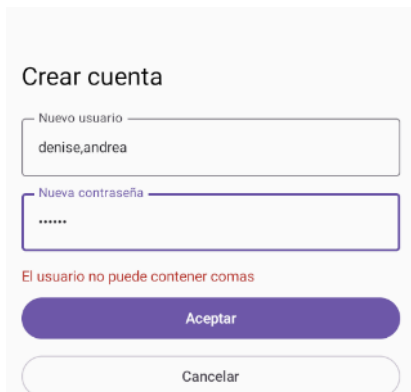


Figura 19: Restricción de caracteres no permitidos (coma) en el nombre de usuario

CUESTIONARIO

CUESTIONARIO

Pregunta 1: ¿Qué ventajas tiene usar Navigation Compose (una sola Activity con varias pantallas) frente al enfoque tradicional de múltiples Activities comunicadas por Intent?

Las principales ventajas son:

- Transiciones más fluidas: al no destruir y recrear Activities, las animaciones entre pantallas son más rápidas y se pueden personalizar.
- Compartición de estado: el estado se comparte fácilmente entre pantallas ya que todas comparten el mismo scope de la Activity.
- Back stack más flexible: Navigation gestiona la pila de retroceso de forma más controlable.
- Patrón unificado: todo se desarrolla en Compose, sin alternar entre intents y callbacks.
- Mejor rendimiento: no hay sobrecarga de crear o destruir múltiples Activities.

Pregunta 2: Además de los argumentos de navegación, ¿qué otro mecanismo podrías usar para compartir datos entre pantallas en una app Compose (por ejemplo, un ViewModel compartido)? Explica brevemente cómo funcionaría.

Un ViewModel compartido es el mecanismo principal. Se crea un ViewModel único y se comparte entre pantallas a través del compositor usando un scope compartido como la Activity:

```
val viewModel: MiViewModel = viewModel()
```

Ventajas:

- Estado centralizado: toda la información compartida se encuentra en un solo lugar, evitando inconsistencias.
- Persistencia: el ViewModel sobrevive a cambios de configuración como rotaciones de pantalla.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

- Desacoplamiento: las pantallas no necesitan conocerse entre sí; solo acceden al ViewModel.
- Reactividad: al usar State o LiveData dentro del ViewModel, los cambios se reflejan automáticamente en todas las pantallas que observen esos estados.

CONCLUSIONES

CONCLUSIONES

Conclusiones

Conclusión 1: Navigation Compose nos permite gestionar varias pantallas dentro de una sola Activity. El NavController controla los desplazamientos y el NavHost centraliza las rutas como login, registro y home/{usuario}, esto hace que el flujo de la aplicación sea más sencillo frente al uso de múltiples Activities comunicadas por Intent.

Conclusión 2: El paso de datos usando la ruta home/{usuario}, declarada con navArgument, permitió enviar el nombre del usuario a HomeScreen. Además, Uri.encode evitó rutas inválidas con caracteres especiales y popUpTo impidió volver al formulario de inicio de sesión con el botón Atrás.

Conclusión 3: Las pantallas @Composable reciben lambdas (onLoginExitoso, onIrARegistro, onRegistroExitoso, onCancelar) en lugar de manejar la navegación por sí mismas, lo que las hace independientes y reutilizables. Por otro lado, remember y mutableStateOf nos permiten que la interfaz conserve lo que escribe el usuario y se actualice sola.

Conclusión 4: El uso de openFileOutput en modo Context.MODE_APPEND permitió persistir las cuentas en cuentas.txt, dentro del almacenamiento interno privado de la aplicación, sin borrar los registros anteriores. La lectura con readLines() y split(", ", limit = 2) permitió validar el ingreso, incluso cuando el archivo aún no existe.

Conclusión 5: Las validaciones de campos vacíos y de caracteres no permitidos, junto con los mensajes de Toast y el texto en color de error, dieron al usuario retroalimentación inmediata. Herramientas como Logcat facilitaron verificar que los datos realmente se guardaban.

Recomendaciones

Recomendación 1: En lugar de guardar la información en un archivo de texto plano (.txt), se sugiere usar herramientas como DataStore, ya que estas permiten organizar, consultar y proteger mucho mejor los datos de la aplicación.

Recomendación 2: No almacenar las contraseñas en texto plano. Es recomendable guardar únicamente un *hash* con *salt* de la contraseña, de modo que si alguien accede al archivo no pueda leer las credenciales.

Recomendación 3: Ampliar las validaciones del registro: verificar que el usuario no exista previamente, ya que actualmente se pueden agregar cuentas duplicadas, exigir una longitud mínima de contraseña y comprobar el formato de los datos antes de guardarlos.

Recomendación 4: Separar los componentes @Composable en archivos Kotlin independientes (LoginScreen.kt, RegistroScreen.kt, HomeScreen.kt) y usar el ViewModel para manejar el estado de la sesión, con el fin de mantener un código más ordenado y fácil de mantener.

Recomendación 5: Es recomendable incluir control de excepciones (try-catch) al leer y escribir archivos, para prevenir fallos inesperados si el archivo no existe, está dañado o no se puede escribir.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

REFERENCIAS
Bibliography Android Developers, "Navigation con Compose". Disponible en: https://developer.android.com/develop/ui/compose/navigation "Navegación en Jetpack Compose con Navigation Compose", video. Disponible en: https://www.youtube.com/watch?v=nwdc1ct06TI